

Technical Implementation Specification

SEASONAL HOBBY HUB ARCHITECTURE & SYSTEMS DESIGN (OPENROUTER UPDATE)

1. Executive Architectural Overview

The Seasonal Hobby Hub is architected as a distraction-free, high-performance web application designed to counter decision paralysis and alleviate the cognitive friction associated with rotating multi-domain hobbies. Acting as a secondary digital brain, the software environment transitions fluidly across visual states while preserving historical context logs.

To ensure near-instantaneous load times, secure state hydration, and an intelligent ambient planning interface, the core technical stack comprises:

- **Frontend Architecture:** Next.js (React) leveraging the App Router paradigm to benefit from server-side rendering (SSR), optimized build layouts, and localized API routing.
- **Styling & Design Token Systems:** Tailwind CSS utilizing a root-level context selector to map seasonal semantic visual variables seamlessly.
- **Database Engine:** Supabase PostgreSQL relational database instance, taking advantage of Row-Level Security (RLS) policies, operational indices, and real-time listeners.
- **AI Compute Layer:** OpenRouter API proxy infrastructure, targeting the reasoning-enabled `poolside/laguna-xs-2.1:free` model node for high-efficiency structured task breakdown and contextual planning.

2. Relational Database Design & Schema

The storage topology relies strictly on explicit data constraints, maintaining data integrity while tracking user sessions, current states, and sequential context changes.

2.1 Database Tables Architecture

Table Name	Field	Data Type	Constraints / Indices
hobbies	id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
	user_id	UUID	FOREIGN KEY -> auth.users(id) ON DELETE CASCADE
	season	season_type (ENUM)	NOT NULL, DEFAULT 'summer' (Index optimized)
	category	VARCHAR(50)	NOT NULL (Index optimized for filtering)
	last_brain_dump	TEXT	Nullable
	is_daily_focus	BOOLEAN	DEFAULT FALSE
activity_logs	id	UUID	PRIMARY KEY
	hobby_id	UUID	FOREIGN KEY -> hobbies(id) ON DELETE SET NULL
	completed_at	DATE	DEFAULT CURRENT_DATE (Index optimized)
	micro_goal_completed	TEXT	NOT NULL

2.2 PostgreSQL Schema Definition (DDL)

```
-- Initialize custom system enums
CREATE TYPE season_type AS ENUM ('summer', 'winter');

-- Primary Hobbies Registry
CREATE TABLE hobbies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
  title VARCHAR(100) NOT NULL,
  category VARCHAR(50) NOT NULL,
  season season_type NOT NULL DEFAULT 'summer',
  icon VARCHAR(50) DEFAULT 'gamepad',
  color_theme VARCHAR(30) DEFAULT 'purple',
  last_brain_dump TEXT,
  micro_goal TEXT,
  notes TEXT,
  is_daily_focus BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Historical Activity Log Ledger
CREATE TABLE activity_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
  hobby_id UUID REFERENCES hobbies(id) ON DELETE SET NULL,
  hobby_title VARCHAR(100) NOT NULL,
  completed_at DATE DEFAULT CURRENT_DATE NOT NULL,
  brain_dump_snapshot TEXT,
  micro_goal_completed TEXT
);

-- Optimization Performance Indices
CREATE INDEX idx_hobbies_user_season ON hobbies(user_id, season);
CREATE INDEX idx_hobbies_category ON hobbies(category);
CREATE INDEX idx_activity_logs_user_date ON activity_logs(user_id, completed_at DESC);
```

3. Frontend State & Semantic Theme Engineering

To switch between the warm-toned Summer dashboard and the cool-toned Winter dashboard dynamically without creating layout shifts, themes are managed at the application root layout using a custom state provider combined with Tailwind CSS custom tokens.

3.1 Theme Implementation Framework

```
/* tailwind.config.js snippet */
module.exports = {
  theme: {
    extend: {
      colors: {
        season: {
          bg: 'var(--season-bg)',
          card: 'var(--season-card)',
          accent: 'var(--season-accent)',
          text: 'var(--season-text)',
        }
      }
    }
  }
}
```

.theme-summer

```
--season-bg: #ffffbf4;
--season-card: #ffffff;
--season-accent: #f39c12;
```

.theme-winter

```
--season-bg: #f4f8fb;
--season-card: #ffffff;
--season-accent: #3498db;
```

4. Core Feature Engineering & Workflows

4.1 The Daily Selector Logic

To mitigate decision paralysis, users are restricted to selecting a maximum of 2 active hobbies for "Today's Focus". The application layer controls this threshold locally before syncing updates to the remote data store:

```
async function handleToggleDailyFocus(hobbyId, currentStatus) {
  if (!currentStatus) {
    const activeCount = await checkActiveDailyFocusCount();
    if (activeCount >= 2) {
      throw new Error("Decision paralysis threshold reached. Maximum 2 items allowed.");
    }
  }
  const { data, error } = await supabase
    .from('hobbies')
    .update({ is_daily_focus: !currentStatus })
    .eq('id', hobbyId);
}
```

4.2 Historical Log Grouping

The app groups history into human-readable formats (e.g., "Monday: Played Zelda"). The standard UI retrieval execution fetches records ordered by date and executes a dynamic group reduction structure:

```
const groupLogsByDay = (logs) => {
  return logs.reduce((acc, log) => {
    const day = new Date(log.completed_at).toLocaleDateString('en-US', { weekday: 'long' });
    if (!acc[day]) acc[day] = [];
    acc[day].push(log.hobby_title);
    return acc;
  }, {});
};
```

5. AI Integration & OpenRouter Orchestration Layer

The AI Assistant functions as a secure contextual secondary brain using the OpenRouter infrastructure. By integrating the reasoning-enabled `poolside/laguna-xs-2.1:free` model, the application handles complex multi-step hobby planning, tracking internal reasoning blocks prior to dispatching responses.

Reasoning Tokens Handling Policy

The implementation utilizes streaming architectures. By capturing data packets dynamically, the frontend strips and renders the `reasoning_details` array separately, allowing users to view the AI's step-by-step contextual breakdown prior to the final micro-goal visualization.

5.1 TypeScript Next.js API Route Integration

```
import { OpenRouter } from "@openrouter/sdk";

export async function POST(req) {
  const { userId, userMessage } = await req.json();

  // Extract user's hobby context array from Supabase data layer
  const { data: hobbies } = await supabase
    .from('hobbies')
    .select('title, category, last_brain_dump, micro_goal')
    .eq('user_id', userId);

  const contextMatrix = hobbies.map(h =>
    `- [${h.category}] ${h.title}: Last Active State: "${h.last_brain_dump}". Micro-Goal: "${h.micro_goal}"`
  ).join('\n');

  const systemInstruction = `
  You are the secondary brain AI helper for a personal hobby dashboard app.
  Current context state of the user's life:
  ${contextMatrix}
  Provide action plans that break down tasks into highly compressed 5-minute low-friction
  micro-goals.
  `;

  const openrouter = new OpenRouter({
    apiKey: process.env.OPENROUTER_API_KEY
  });

  // Stream execution payload containing reasoning capabilities
  const stream = await openrouter.chat.send({
    chatRequest: {
      model: "poolside/laguna-xs-2.1:free",
      messages: [
        { role: "system", content: systemInstruction },
        { role: "user", content: userMessage }
      ],
      stream: true,
      reasoning: {
        effort: "medium"
      }
    }
  });

  // Handling and piping implementation continues downstream to client...
}
```

5.2 OpenRouter Operational Reference Matrix

HTTP Method	Endpoint Path	Authorization	Optional Leaderboard Headers
POST	https://openrouter.ai/api/v1/chat/completions	Bearer \$OPENROUTER_API_KEY	HTTP-Referer , X-Title

6. Deployment, Environmental Variables & Production Scaling

The deployment architecture maps out serverless components targeting Vercel's Edge and Serverless runtimes. Because AI connections are routed through OpenRouter over secure API sockets, backend computation overhead remains flat, allowing extreme production scalability.

6.1 Connection Optimization

- **Supabase Database Connectivity:** Database connections must utilize connection pooling variables (Session-mode transaction strings for serverless functions, Direct execution strings strictly for long-running synchronization workflows).
- **Serverless Stream Timeouts:** Next.js Route Handlers interfacing with OpenRouter streams should utilize explicit Edge runtime runtime constraints to avoid cold start execution ceilings on Vercel's basic plans.

6.2 Environment Security Configuration

The production environment workspace relies strictly on asymmetric configuration keys configured inside the Vercel dashboard:

Environment Key Variable	Visibility Tier	Functional Operational Purpose
NEXT_PUBLIC_SUPABASE_URL	Public Client Space	Public target database client endpoint access URL.
NEXT_PUBLIC_SUPABASE_ANON_KEY	Public Client Space	Anonymous client authorization key safe for standard browser asset compilation.
SUPABASE_SERVICE_ROLE_KEY	Private Server Only	Strictly secret bypass credential kept completely out of client-side bundles.
OPENROUTER_API_KEY	Private Server Only	Secret proxy credential required to authenticate and process poolside/laguna-xs-2.1:free model turns.